



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 实验 4: 二叉查找树的实现

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 19 日

实验报告

【实验目的】

掌握二叉查找树的基本概念及其基本操作的实现；

【实验原理】

二叉查找树（BST）是一种利用二叉树结构进行对一系列数据进行查找的一种数据结构，其定义结构为：

- (1) 根节点
- (2) 左子树根节点小于根节点
- (3) 右子树根节点大于根节点

那么，以此为规则，二叉查找树的插入操作便可以很好实现了：

- (1) 大于根节点->递归插右子树
- (2) 小于根节点->递归插左子树

由书上所说，对于二叉查找树，如果我们想要递增输出节点，我们只要对查找二叉树进行中序遍历即可。中序遍历定义如下：

- (1) 遍历左子树
- (2) 遍历根节点
- (3) 遍历右子树

我们应该使用递归调用的方法来构建中序遍历函数，终止条件定为节点为空即可。

如果我们想要递减输出，那么逆中序遍历即可。

对于查找函数，我们同样可以通过递归调用来解决函数的实现，类似插入查找二叉树的实现：

- (1) 根节点等于查找值，输出 1
- (2) 根节点小于查找值，查找其右子树
- (3) 根节点大于查找值，查找左其子树

【实验要求】

输入数据的第一行为一个正整数T，表示测试数据的组数，即共T组测试数据。每组测试数据的第一行输入正整数n($5 \leq n \leq 20$)，第二行输入n个整数，要求依次完成以下工作:

- 1) 以这n个整数生成一棵用链式存储结构存储的二叉排序树;
- 2) 按递增顺序输出该二叉排序树中所有数;
- 3) 输入一个整数key，对该二叉排序树进行查找，若在该二叉排序树中存在这个整数key，则输出find，否则输出not find;
- 4) 输入一个整数key，若该二叉排序树中不存在这个整数key，则将key插入到该二叉排序树中，使插入后仍为原性质的二叉排序树;否则不必插入;
- 5) 在 4)的基础上，按递减顺序输出该二叉排序树中的所有数。

- 1) 实现二叉查找树的定义;
- 2) 实现二叉查找树的查找、插入操作

【实验源程序】

1.bitree 头文件

```
#ifndef _bitree_H
#define _bitree_H

#define Maxsize 255
//树节点结构
typedef struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;

TreeNode* createNode(int data); //创建节点
int isEmpty_T(TreeNode* bintree); //判断树是否为空
TreeNode* insert(TreeNode* root, int key); //插入查找二叉树节点
```

```

void levelOrderTraversal(TreeNode* root); // 层次遍历
int search(TreeNode* root, int key); // 查找节点
void inOrderTraversal(TreeNode* root, int* arr, int* index); // 中序遍历
void reverseInOrderTraversal(TreeNode* root, int* arr, int* index); // 反中序遍历
void freeTree(TreeNode* root); // 释放树

#endif
    
```

2.bitree 函数文件

```

#include<stdio.h>
#include <stdlib.h>
#include "bitree.h"
#include "ADT.h"

/*
 * 创建新树节点
 * 参数 data: 节点存储的数据
 * 返回值: 新创建的节点指针, 左右子节点初始化为 NULL
 */
TreeNode* createNode(int data)
{
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

/*
 * 参数 bintree: 待判断的树节点指针
 * 返回值: 数据为空返回 0, 否则返回 1
 */
int isEmpty_T(TreeNode* bintree) {
    if(bintree->data == NULL)
        return 0;
    return 1;
}
    
```

```

}

/*
 * 向二叉搜索树插入节点（递归实现）
 * 参数 root: 树根节点指针
 * 参数 key: 要插入的数据
 * 返回值: 插入后的新根节点指针
 */
TreeNode* insert(TreeNode* root, int key) {
    if (root == NULL) {
        TreeNode* newNode = (TreeNode*)malloc(sizeof(TreeNode));
        newNode->data = key;
        newNode->left = newNode->right = NULL;
        return newNode;
    }
    if (key < root->data) {
        root->left = insert(root->left, key);
    } else if (key > root->data) {
        root->right = insert(root->right, key);
    }
    return root;
}

/*
 * 二叉搜索树查找节点（递归实现）
 * 参数 root: 树根节点指针
 * 参数 key: 要查找的数据
 * 返回值: 存在返回 1, 否则返回 0
 */
int search(TreeNode* root, int key) {
    if (root == NULL) return 0;
    if (key == root->data) return 1;
    if (key < root->data) return search(root->left, key);
    else return search(root->right, key);
}

/*
 * 中序遍历（左根右），结果存入数组

```

```

    * 参数 root: 树根节点指针
    * 参数 arr: 用于存储结果的数组
    * 参数 index: 数组当前存储位置的指针
    */
void inOrderTraversal(TreeNode* root, int* arr, int* index) {
    if (root == NULL) return;
    inOrderTraversal(root->left, arr, index);
    arr[(*index)++] = root->data;
    inOrderTraversal(root->right, arr, index);
}

/*
    * 逆中序遍历（右根左），结果存入数组（降序）
    * 参数 root: 树根节点指针
    * 参数 arr: 用于存储结果的数组
    * 参数 index: 数组当前存储位置的指针
    */
void reverseInOrderTraversal(TreeNode* root, int* arr, int* index) {
    if (root == NULL) return;
    reverseInOrderTraversal(root->right, arr, index);
    arr[(*index)++] = root->data;
    reverseInOrderTraversal(root->left, arr, index);
}

/*
    * 递归释放二叉树内存
    * 参数 root: 树根节点指针
    */
void freeTree(TreeNode* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

/*
    * 层次遍历（广度优先遍历）
    * 参数 bintree: 树根节点指针

```

```

    */
void LevelTraversal(TreeNode* bintree){
    TreeNode* q;
    TreeNode* T[Maxsize];

    int front,rear;
    rear = -1;
    front = -1;

    if(isEmpty_T(bintree)== 0)
        return;

    T[0] = bintree;
    rear++;

    while(front != rear) {
        front++;
        q = T[front];
        printf("%d ",q->data);

        if(q->left != NULL){
            rear++;
            T[rear] = q->left;
        }

        if(q->right != NULL){
            rear++;
            T[rear] = q->right;
        }
    }
    return;
}
    
```

3.主程序应用示例

```

#include<stdio.h>
#include "bitree.h"
    
```

```

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int n;
        scanf("%d", &n);
        TreeNode* root = NULL;
        //构造查找二叉树
        for (int i = 0; i < n; ++i) {
            int num;
            scanf("%d", &num);
            root = insert(root, num);
        }

        // 按递增顺序输出
        printf("正序输出: \n");
        int arr1[20];
        int size1 = 0;
        inOrderTraversal(root, arr1, &size1);
        for (int i = 0; i < size1; ++i) {
            if (i > 0) printf(" ");
            printf("%d", arr1[i]);
        }
        printf("\n");

        // 查找 key1
        int key1;
        scanf("%d", &key1);
        if (search(root, key1)) {
            printf("find\n");
        } else {
            printf("not find\n");
        }

        // 入 key2
        int key2;
        scanf("%d", &key2);
        root = insert(root, key2);
    }
}
    
```

```

        printf("倒叙输出: \n");
        // 按递减顺序输出
        int arr2[21]; // 插入后最多 21 个元素
        int size2 = 0;
        reverseInOrderTraversal(root, arr2, &size2);
        for (int i = 0; i < size2; ++i) {
            if (i > 0) printf(" ");
            printf("%d", arr2[i]);
        }
        printf("\n");

        // 释放当前树的内存
        freeTree(root);
        root = NULL;
    }
    return 0;
}

```

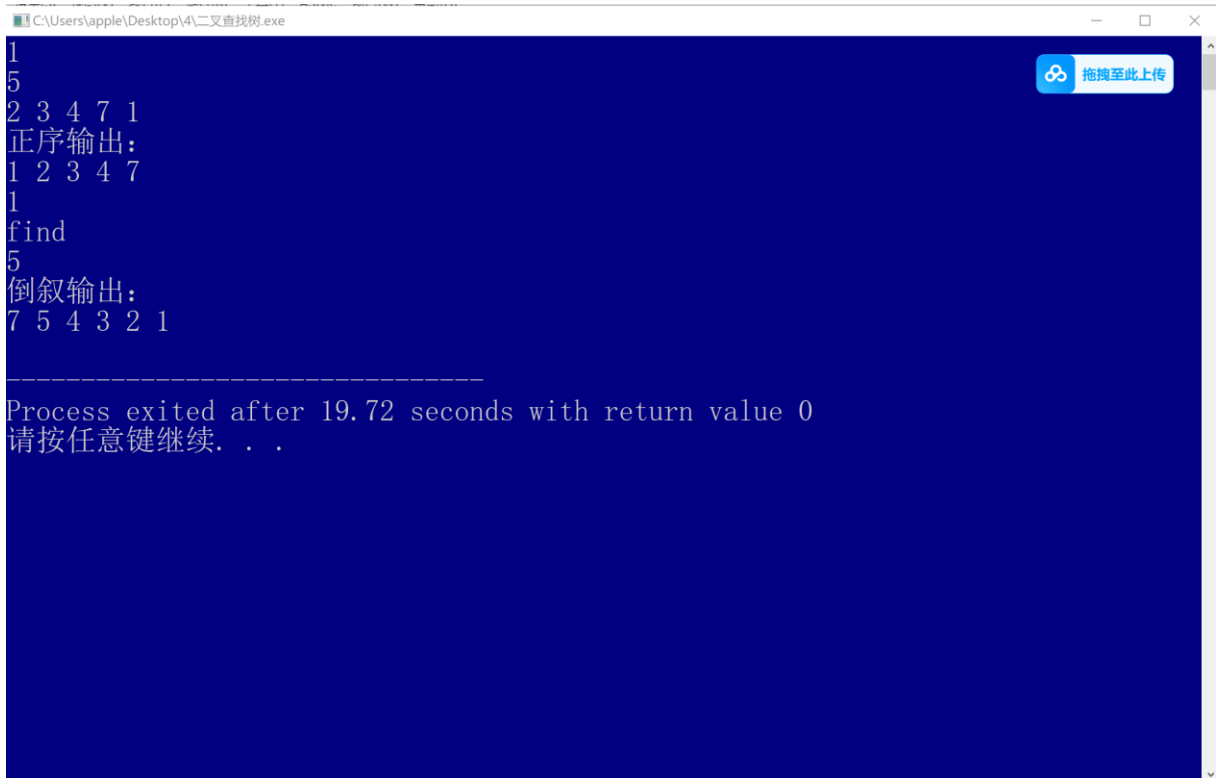
【实验结果】

输入数据：

```

1
5
2 3 4 7 1
2
5

```



```

C:\Users\apple\Desktop\4\二叉查找树.exe
1
5
2 3 4 7 1
正序输出:
1 2 3 4 7
1
find
5
倒叙输出:
7 5 4 3 2 1

-----
Process exited after 19.72 seconds with return value 0
请按任意键继续. . .
    
```

图 1 运行结果截图

可以发现，当我们输入 2, 3, 4, 7, 1 时，输出为递增输出；当我们查找数据 1 时，显示可以查找到；当我们选择插入并倒叙输出时，显示插入成功且倒序输出，与预期的实验结果符合得非常好，实验成功。

【实验心得】

本次实验我实现了经典二叉查找树的基本结构以及其基本操作如：插入，查找等，并按照实验要求的示例完成了实验，我们均是通过指针与递归实现有关的所有的算法的，这根本上是由于树天然是一个递归结构，每一个节点都是相似的，因此可以分解成非常多的子问题更容易，通过递归来简洁描述。

上述描述也只适用于树规模较小的情况，当树深度非常非常大时 (10^5)，此时就不能通过递归描述了，应该使用队列来描述，利用迭代完成算法。(如实验 3 中的层次化遍历)

同时，在过程中我也遇到了一些问题，可能需要后续的改进：

对于重复元素的处理

如书上所说，经典的 BST 对于重复元素是不处理的，它们不会被重复地插入树中，但是在

实际应用中，我们有可能需要知道重复元素的个数，因此，我们可以在基本二叉查找树这一数据结构中加上如下变量：

```
typedef struct TreeNode {  
    int data;  
    int num;  
    struct TreeNode* left;  
    struct TreeNode* right;  
} TreeNode;
```

对于插入重复元素，我们将对应的 num 变量自增即可。